



NGO Connect

Customer Support Strategy

Phase-wise Implementation Guide

Phase 1 (MVP) → Phase 2 (Help Desk) → Phase 3 (AI Support)

Document Information

Version:	v1.0 — Draft
Date:	July 2026
Status:	For Internal Use
Platform:	RippleHub (NGO Connect)
Audience:	Product / Development / Operations

1. Executive Summary

NGO Connect (RippleHub) serves a multi-sided ecosystem — volunteers, NGO admins, donors, and super admins. Each group encounters different issues, carries different urgency levels, and expects a different quality of response. This document defines a **three-phase support strategy** designed to grow alongside the platform: start lean with in-app forms and email, scale to a structured help desk with ticket IDs and SLA tracking, and ultimately reach AI-powered self-service that deflects the majority of tickets automatically.

The core principle is **pragmatic escalation** — never over-engineer support infrastructure ahead of need. Email handles Phase 1 adequately. Investing in a ticketing system before you have ticket volume is pure waste. This document gives you exact trigger criteria to know when to move between phases, along with complete implementation checklists, DB schemas, API specs, and a risk log.

Phase Overview

Phase	Trigger Criteria	Support Channels	Backend
Phase 1 MVP	0–100 NGOs (Start now)	In-app form WhatsApp link Email inbox	support@ripplehub.app (Gmail / Workspace)
Phase 2 Help Desk	100+ NGOs or 50+ tickets/month	Phase 1 + Ticket IDs Auto-acknowledgement	Freshdesk (REST API integration)
Phase 3 AI Support	1,000+ NGOs or 500+ tickets/month	Phase 2 + AI chatbot Self-service KB	Chatbot + Freshdesk + Knowledge Base

Table 1 — Phase trigger criteria and channel summary

2. Who Contacts Support and Why

Four distinct user types will contact support. Their pain points, urgency levels, and expected response quality are fundamentally different. This mapping drives the category structure, SLA matrix, and email routing rules across all three phases.

User Type	Common Issues	Priority	Est. Volume
Donors	Payment failed, receipt not received, 80G cert missing, refund request	P1 Critical Same-day	Low volume Highest urgency
NGO Admins & Founders	Approval status, rejection reason, resubmission help, member management	P2 High 24-hour	Medium volume Medium urgency
Volunteers & Users	OTP not received, cannot join project, profile issue, app crash/bug	P2/P3 Medium 24-48 hour	Highest volume Lower urgency
Super Admin	Platform config, bulk actions, escalated complaints from NGOs	Internal (no public SLA)	Very low Internal only

Table 2 — User type support matrix

3. Support Categories & SLA Matrix

Five categories map directly to the **LookupType SUPPORT_CATEGORY** in the database. Each has a defined priority, target first-response time (Phase 1 = internal goal, Phase 2+ = enforced by Freshdesk SLA policies), and team owner.

Category (ValueCode)	Example Issues	Priority	Response P1	Response P2+	Owner
DONATION Donation Support	Payment failed, receipt missing, 80G cert, refund, Razorpay dispute	P1 Critical	4 hrs	2 hrs	Finance
ORG_APPROVAL Org Approval	Approval status, rejection reason, resubmission, document issue	P2 High	24 hrs	8 hrs	Admin Team
BUG_REPORT Bug / App Issue	OTP not received, upload fail, app crash, login stuck	P2 High	12 hrs	4 hrs	Dev Team
GENERAL General Query	How-to questions, feature queries, navigation, account settings	P3 Normal	48 hrs	24 hrs	Support
FEEDBACK Feedback / Idea	Feature requests, UX suggestions, compliments, complaints	P4 Low	5 days	3 days	Product

Table 3 — Support category SLA matrix

Phase 1 — MVP Support

Trigger: 0–100 NGOs | Now → Until 50+ tickets/month

Overview

Phase 1 is intentionally lean. No external ticketing system, no paid tools, no SupportTickets database table. Every request routes to **support@ripplehub.app** via the platform's existing email provider. The team triages manually. This is adequate until approximately 50 tickets per month — at that point manual overhead justifies migrating to Phase 2.

4.1 What to Build

- **Help & Support screen** — New entry in Profile → Settings (above Account Settings). Routes to a dedicated support form screen.
- **In-app support form** — Category chip selector (5 options), Subject (max 120 chars), Description (max 1,000 chars). Name and email pre-filled from user profile; editable.
- **POST /api/v1/support/contact** — No DB write. SP logs to AuditLogs for traceability, then signals the DAL to dispatch an email via IEmailProvider.
- **WhatsApp Business link** — "Chat on WhatsApp" button below the form. Opens wa.me deep link with pre-filled message. No WhatsApp Business API required in Phase 1.
- **Contextual help banners** — Small info strips on 4 key screens where user confusion peaks. Each banner links to the form pre-filled with the correct category.

4.2 Screen Components

Component	Type	Validation / Notes
Category	Chip / Dropdown selector	Required. 5 values from SUPPORT_CATEGORY lookup.
Subject	TextInput (1 line)	Required. Max 120 chars. Show char count.
Description	TextInput (multiline, 5 rows)	Required. Min 20 chars, max 1,000 chars.
Contact Email	TextInput	Pre-filled from profile. Editable. Format-validated.
Attachment	Image picker	Phase 2 only — skip in Phase 1.
Submit button	Pressable	Disabled while loading. Success: toast + confirmation screen.
WhatsApp CTA	TouchableOpacity	Secondary button below submit. Opens wa.me deep link.

Table 4 — Help & Support screen components (Phase 1)

4.3 API & SP Specification

Endpoint: POST /api/v1/support/contact

Field	Type	Required	Notes
categoryCode	string	Yes	ValueCode from SUPPORT_CATEGORY LookupType
subject	string	Yes	Max 120 chars
description	string	Yes	Min 20 chars, max 1,000 chars
contactEmail	string	Yes	Defaults to user profile email
contactName	string	No	Defaults to user full name from profile
screenContext	string	No	Which screen triggered the form (analytics)

Table 5 — Request body for POST /api/v1/support/contact

SP: Support_SendContactEmail

```

CREATE PROCEDURE Support_SendContactEmail(
    IN p_UserId          INT UNSIGNED,
    IN p_CategoryCode    VARCHAR(50),
    IN p_Subject         VARCHAR(120),
    IN p_Description     TEXT,
    IN p_ContactEmail    VARCHAR(150),
    IN p_ContactName     VARCHAR(100),
    IN p_ScreenCtx       VARCHAR(100)
)
BEGIN
    INSERT INTO AuditLogs (UserId, Action, EntityName, EntityId, NewValue, CreatedAt)
    VALUES (p_UserId, 'SUPPORT_CONTACT', 'SupportRequest', 0,
        JSON_OBJECT('category', p_CategoryCode, 'subject', p_Subject), NOW());
    SELECT 1 AS IsSuccess,
        'Your request has been received. We will respond shortly.' AS Message,
        p_CategoryCode AS CategoryCode, p_ContactEmail AS ContactEmail,
        p_ContactName AS ContactName, p_Subject AS Subject,
        p_Description AS Description;
END

```

Note: No SupportTickets row in Phase 1. Only an AuditLog entry for traceability. Email dispatch is handled in SupportDal.cs after reading the SP result row via IEmailProvider. Zero schema changes beyond one SP and one AuditLogs INSERT.

4.4 Email Routing Setup

The category code is injected into the email subject as a prefix (e.g. "[DONATION] Payment failed"). Set up Gmail filters on support@ripplehub.app to auto-label and route by this prefix.

Subject Prefix	Gmail Label	Auto-Star	Routed To
[DONATION]	Support/Donation	Yes ★	Finance contact
[ORG_APPROVAL]	Support/OrgApproval	Yes ★	Admin team
[BUG_REPORT]	Support/Bugs	No	Dev on-call
[GENERAL]	Support/General	No	Rotating support
[FEEDBACK]	Support/Feedback	No	Product manager

Table 6 — Gmail routing rules for Phase 1

4.5 WhatsApp Business Setup

- Register a **dedicated number** (not the founder's personal phone) with WhatsApp Business.
- Set up the Business profile: name = RippleHub, category = Technology, website = ripplehub.app.
- Configure **Away Message** for outside business hours: "We've received your message and will respond within 24 hours."
- In-app deep link: <https://wa.me/91XXXXXXXXXX?text=Hi%2C+I+need+help+with+NGO+Connect>
- WhatsApp Business API is **not needed in Phase 1** — only required when you need automated replies or chatbot routing (Phase 3).

4.6 Contextual Help Banners

Instead of a generic FAQ page (which goes stale and nobody reads), place small contextual help banners at the exact screens where confusion peaks. Each banner links directly to the support form pre-filled with the relevant category.

Screen (Prototype ID)	Banner Text	Pre-fill Category
Org Approval Pending (s-my-orgs)	Approval typically takes 3–5 business days. Questions? Contact us →	ORG_APPROVAL
Donation Payment Failed (s-donation-payment)	Payment did not go through? We'll help. Contact support →	DONATION
OTP Screen (s-otp)	Didn't get the OTP? Wait 60 s, then retry. Still stuck? Contact us →	BUG_REPORT
NGO Rejection Notice (email / in-app notifications)	Read the rejection reason above, then resubmit or Contact us →	ORG_APPROVAL

Table 7 — Contextual help banner placement

4.7 Phase 1 Implementation Checklist

- 1. Add "Help & Support" entry to ProfileScreen SETTINGS_ITEMS (above Account Settings)
- 2. Create HelpSupportScreen.tsx — category chip selector, subject, description, email fields
- 3. Register HelpSupportScreen in AppNavigator.tsx
- 4. Create Support_SendContactEmail SP in setup SQL + patch file
- 5. Create ISupportDal interface + SupportDal.cs + SupportController.cs
- 6. Implement POST /api/v1/support/contact — SP call → email dispatch via IEmailProvider
- 7. Configure support@ripplehub.app Gmail labels and filter rules (Table 6)
- 8. Register dedicated WhatsApp Business number — set business hours + away message
- 9. Add contextual help banners on 4 key screens (see Table 7)
- 10. Write and publish Refund & Dispute Policy page (Razorpay compliance requirement)

Phase 2 — Help Desk

Trigger: 100+ NGOs or 50+ tickets/month | Freshdesk integration

Overview

When manual email triage consumes 1+ hours per day, move to a structured help desk. **Freshdesk** is recommended: Indian company (Freshworks), free tier for up to 10 agents, full REST API, native WhatsApp integration roadmap, and built-in Freddy AI for Phase 3. This phase also introduces the **SupportTickets** table, giving users an in-app "My Support Requests" screen with real ticket IDs and status tracking.

5.1 Tool Comparison

Criterion	Freshdesk	Zendesk	Intercom
Free tier	10 agents free forever	Trial only	Trial only
India billing	INR billing, affordable	USD only, expensive	USD only, expensive
REST API	Full CRUD API	Full API	Full API
WhatsApp (native)	Yes (Phase 3 ready)	Available (add-on)	Limited
Built-in AI	Freddy AI (included)	Zendesk AI (paid)	Fin AI (expensive)
Setup effort	Low — 2–3 hours	Medium	Medium–High

Table 8 — Help desk tool comparison

5.2 Database Schema (SupportTickets)

Phase 2 introduces the **SupportTickets** table — separate from AuditLogs (which remain a write-only trace). SupportTickets enables user-facing ticket history and gives super admins a query surface for support reporting.

```
CREATE TABLE SupportTickets (
  TicketId          INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  UserId            INT UNSIGNED NOT NULL,
  CategoryLkpId     INT UNSIGNED NOT NULL,    -- LookupType: SUPPORT_CATEGORY
  Subject           VARCHAR(120) NOT NULL,
  Description        TEXT          NOT NULL,
  ContactEmail      VARCHAR(150)  NOT NULL,
  StatusCode        VARCHAR(20)   NOT NULL DEFAULT 'OPEN',
                                     -- OPEN|IN_PROGRESS|RESOLVED|CLOSED
  ExternalTicketId  VARCHAR(50)   NULL,       -- Freshdesk ticket ID
  ExternalUrl       VARCHAR(500)  NULL,       -- Freshdesk ticket URL
  ResolutionNote    TEXT          NULL,
  CreatedAt         DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UpdatedAt         DATETIME      NULL ON UPDATE CURRENT_TIMESTAMP,
  ResolvedAt        DATETIME      NULL,
  IsDeleted         TINYINT(1)    NOT NULL DEFAULT 0,
  FOREIGN KEY (UserId) REFERENCES Users(UserId),
  FOREIGN KEY (CategoryLkpId) REFERENCES LookupValues(LookupValueId),
  INDEX idx_UserId   (UserId),
  INDEX idx_Status   (StatusCode),
  INDEX idx_CreatedAt (CreatedAt)
);
```

SUPPORT_CATEGORY LookupValues Seed Data

ValueCode	ValueName	OrderNo	IsDefault	IsSystemValue
DONATION	Donation Support	1	0	1
ORG_APPROVAL	Organisation Approval	2	0	1
BUG_REPORT	Bug / App Issue	3	0	1
GENERAL	General Query	4	1	1
FEEDBACK	Feedback / Suggestion	5	0	1

Table 9 — SUPPORT_CATEGORY LookupValues seed data

5.3 Phase 2 API Endpoints

Endpoint	Method	Purpose
/api/v1/support/tickets	POST	Create ticket → writes SupportTickets row + creates Freshdesk ticket via API
/api/v1/support/tickets	GET	List current user's own tickets (paginated, CreatedAt DESC)
/api/v1/support/tickets/{id}	GET	Get single ticket status + resolution note
/api/v1/support/webhook/freshdesk	POST	Freshdesk webhook → update StatusCode + ResolvedAt in SupportTickets

Table 10 — Phase 2 API endpoints

5.4 Freshdesk Integration Notes

- Store Freshdesk API key in **Settings table**: SettingGroup=SUPPORT, SettingKey=FRESHDESK_API_KEY, IsPublic=0. Read from SettingsCache — zero DB calls.
- SupportDal.CreateTicketAsync(): (1) write SupportTickets row → (2) POST to Freshdesk API → (3) update row with ExternalTicketId.
- Set SLA policies in Freshdesk dashboard matching Table 3 response targets. Freshdesk sends breach alerts automatically.
- Freshdesk webhook fires on ticket status change → your /webhook/freshdesk endpoint updates StatusCode and ResolvedAt in SupportTickets.
- In-app "My Support Requests" tab shows ticket history using Support_GetMyTickets SP — paged list, most recent first.

Settings Table Entries Required (Phase 2):

SUPPORT / FRESHDESK_API_KEY — Freshdesk API key (IsPublic=0)
 SUPPORT / FRESHDESK_DOMAIN — e.g. ripplehub.freshdesk.com (IsPublic=0)
 SUPPORT / FRESHDESK_ENABLED — true/false feature flag (IsPublic=0)
 SUPPORT / WHATSAPP_NUMBER — e.g. +91XXXXXXXXXX (IsPublic=1)

5.5 Phase 2 Implementation Checklist

- 1. Sign up at freshdesk.com — free tier (10 agents) is sufficient initially
- 2. Create Freshdesk categories matching SUPPORT_CATEGORY LookupType values
- 3. Set Freshdesk SLA policies matching Table 3 response targets
- 4. Add SupportTickets table to setup SQL + patch file (DDL in Section 5.2)
- 5. Add LookupType SUPPORT_CATEGORY with 5 seed values (Table 9)
- 6. Add 4 new Settings rows for Freshdesk config (see info box above)
- 7. Update SupportDal.cs: CreateTicketAsync() — DB write + Freshdesk API call
- 8. Create Support_CreateTicket SP (write SupportTickets, return TicketId)
- 9. Create Support_GetMyTickets SP (paginated, user-scoped)

- 10. Add "My Requests" tab to HelpSupportScreen.tsx
- 11. Create POST /api/v1/support/webhook/freshdesk endpoint
- 12. Configure Freshdesk to call webhook on ticket status change
- 13. Add attachment upload to support form (image picker → /media/upload)
- 14. Schedule data retention cleanup job — anonymise tickets older than 2 years

Phase 3 — AI-Powered Support

Trigger: 1,000+ NGOs or 500+ tickets/month | AI chatbot + self-service KB

Overview

Phase 3 aligns directly with the NGO Connect AI Integration Roadmap (Phase 1: NGO Help Chatbot). The chatbot is **not** a third-party widget — it is a native in-app screen that calls the NGO Connect backend, which injects real-time user context (approval status, donation history, ticket history) before calling the LLM. Industry benchmark: a well-trained chatbot deflects **40–60% of inbound tickets**. For an NGO platform with predictable query types, 50% deflection is a realistic 6-month target.

6.1 Architecture

Layer	Component	Responsibility
Mobile	ChatSupportScreen.tsx	Chat UI — message list, input bar, "Talk to a human" CTA
API	POST /api/v1/support/chat	Receive message, inject user context, call LLM, return reply
LLM	Claude API / GPT-4o	Generate response with system prompt + injected context
Escalation	Auto-ticket on intent detect	Detect "human/agent/escalate" intent → create Freshdesk ticket
Knowledge	KnowledgeBase table	Curated Q&A pairs seeded from resolved Freshdesk tickets
Analytics	ChatAnalytics table	Log deflection outcome, CSAT score, unresolved query patterns

Table 11 — AI chatbot architecture layers

6.2 Chatbot Capability Matrix

Category	AI Handles (Tier-0)	Escalate to Human (Tier-1)
DONATION	Show receipt, payment status, 80G policy, refund policy	Account verification request, Razorpay dispute, partial payment
ORG_APPROVAL	Approval status, rejection reason, resubmission	Approval exception, document re-verification, appeal
BUG_REPORT	Troubleshoot OTP, cache clear, known workarounds	Known crash, data loss, security issue, unresolved bug
GENERAL	How-to answers, feature explanation, account creation	Complex queries not in knowledge base
FEEDBACK	Acknowledge, auto-log to product backlog, thank you	Rarely escalated — reviewed async by product team

Table 12 — Chatbot Tier-0 vs Tier-1 capability matrix

6.3 Success Metrics

Metric	Definition	Target
Deflection Rate	Tickets resolved by AI without human intervention	50% within 6 months
CSAT (AI)	User satisfaction after AI resolution (1–5 stars)	≥ 4.0 / 5.0
First Response	Time from user message to AI reply	< 3 seconds
Escalation Rate	% of chat sessions escalated to human agent	≤ 25%
Resolution Rate	Tickets fully resolved (no re-open within 72h)	≥ 80%
KB Coverage	% of incoming queries matched to knowledge base	70% by Month 3

Table 13 — Phase 3 success metrics

6.4 Phase 3 Implementation Checklist

- 1. Build ChatSupportScreen.tsx — chat UI, typing indicator, escalate CTA
- 2. Create KnowledgeBase table — seed with 30+ Q&A pairs from resolved Freshdesk tickets
- 3. Create POST /api/v1/support/chat endpoint with context injection + LLM call
- 4. Store LLM API key in Settings: SUPPORT / LLM_API_KEY (IsPublic=0)
- 5. Store LLM model name in Settings: SUPPORT / LLM_MODEL e.g. claude-sonnet-5
- 6. Build auto-escalation logic — detect intent → create Freshdesk ticket → notify user
- 7. Create ChatAnalytics table — log every session, deflection outcome, CSAT
- 8. Integrate Freshdesk Freddy AI (optional) or keep custom LLM integration
- 9. Weekly knowledge base review process — unresolved queries → new KB entries
- 10. Build support analytics dashboard for product/ops team
- 11. Publish help.ripplehub.app with SLA commitments + searchable KB articles

7. Policy & Compliance Requirements

7.1 Razorpay Merchant Compliance

Razorpay's merchant agreement requires a documented **Refund & Dispute Policy** before going live with payments. This is mandatory — not optional.

- Publish Refund Policy at ripplehub.app/refund **before payment go-live**.
- Maximum refund turnaround: **7–10 business days** (standard Razorpay mandate).
- Payment dispute first response: **within 2 business hours** (P1 SLA, Table 3).
- Razorpay Disputes dashboard must be checked **daily** by the finance contact.
- All donation transactions use DON-2026-XXXXXX readable IDs — already implemented.
- 80G receipts must be generated within **7 days** of a qualifying donation.

7.2 Data Privacy on Support Forms

- Support form screen: add note "Your details will be used only to resolve your request and will not be shared with third parties."
- Freshdesk (Phase 2) must have a **Data Processing Agreement (DPA)** signed with Freshworks before storing user data.
- Support ticket data must not be retained beyond **2 years** — build a cleanup Hangfire job in Phase 2.
- Users can request deletion of their support history under DPDP / GDPR rights.
- AuditLogs entries from support contacts must be anonymisable on account deletion.

7.3 Published SLA Commitments

Publish these commitments on help.ripplehub.app (Phase 3) or in the app's Help screen (Phase 2). Do not publish SLAs you cannot currently meet.

Issue Type	Published (User-facing)	Internal Target
Payment / Donation issues	4 hours (business hours)	2 hours
Organisation approval	1 business day	8 hours
App bugs / account issues	1 business day	4 hours
General questions	2 business days	24 hours
Feature suggestions	5 business days	3 days

Table 14 — Published vs internal SLA targets

8. Master Implementation Roadmap

#	Task	Phase	Effort
1	Add Help & Support to ProfileScreen SETTINGS_ITEMS	1	0.5 h
2	Create HelpSupportScreen.tsx (category + form)	1	4 h
3	Register HelpSupportScreen in AppNavigator	1	0.25 h
4	Support_SendContactEmail SP + patch file	1	1.5 h
5	ISupportDal + SupportDal.cs + SupportController.cs	1	2 h
6	POST /api/v1/support/contact endpoint	1	1 h
7	Configure support@ripplehub.app + Gmail label rules	1	0.5 h
8	WhatsApp Business number setup + away message	1	1 h
9	Contextual help banners on 4 screens	1	2 h
10	Publish Refund & Dispute Policy (Razorpay compliance)	1	2 h
11	Freshdesk account + categories + SLA policies	2	3 h
12	SupportTickets table + SUPPORT_CATEGORY LookupType	2	1.5 h
13	Add Freshdesk Settings rows (4 entries)	2	0.5 h
14	Update SupportDal — DB write + Freshdesk API call	2	3 h
15	Support_CreateTicket + Support_GetMyTickets SPs	2	2 h
16	"My Requests" tab in HelpSupportScreen	2	3 h
17	POST /api/v1/support/webhook/freshdesk endpoint	2	2 h
18	Data retention cleanup Hangfire job (2-year limit)	2	2 h
19	Build ChatSupportScreen.tsx (AI chat UI)	3	6 h
20	KnowledgeBase table + 30+ seed Q&A pairs	3	4 h
21	POST /api/v1/support/chat + LLM context injection	3	8 h
22	Auto-escalation from chatbot to Freshdesk	3	3 h
23	ChatAnalytics table + deflection rate dashboard	3	4 h
24	Publish help.ripplehub.app with SLAs + KB articles	3	4 h

Table 15 — Master implementation task list across all phases

Effort Summary: Phase 1 ≈ 15 hours (1 sprint) — Phase 2 ≈ 17 hours (1 sprint) — Phase 3 ≈ 29 hours (2 sprints). Estimates are for one full-stack developer with existing NGO Connect codebase familiarity. Phase 1 should be completed before the donation module goes live.

9. Blind Spots & Risk Log

	Impact	Mitigation
Contacts support for a refund that is actually a Razorpay Refund Policy request. Finance team trained on Razorpay Disputes dashboard before payment.	High	Separate Refund Policy page.
Box overwhelmed before Phase 2 migration is ready.	Medium	Set Phase 2 trigger at 50 tickets/month, not 100 NGOs. Monitor inbox daily from Day 1.
Personal number used before Business account registered.	Low	Register dedicated company SIM before publishing the number in the app.

	Impact	Mitigation
costs spike in Phase 3 if chatbot handles all queue without human responses. Per-user daily message limit. Use a cheaper model tier for Tier-0.	High	Cash with human response
k free tier (10 agents) exceeded as team grows	Low	Budget for Freshdesk Growth plan ~\$15/agent/month at Phase 2 planning.
agent accidentally shares one user's data with another	Medium	Written SOP: agents must never share PII without consent. DPDP compliance.
approval SLA not met during high-volume boarding	Medium	Reassign dedicated admin resource during campaigns. Escalate to super admin if queue > 20.

Table 16 — Risk log with mitigations